

Unit 1: Introduction to XML

Introduction to XML

XML (eXtensible Markup Language) is a markup language designed for storing and transporting data in a structured way. It focuses on data representation, not presentation. Unlike HTML, which has predefined tags for formatting and layout, XML allows users to define custom tags according to the data they need to store.

XML is both human-readable and machine-readable. It plays a crucial role in data exchange between different systems over the web. For example, XML is commonly used in web services (SOAP), RSS feeds, configuration files, and data storage.

Rules:

- XML allows custom tag creation.
- It is platform-independent and text-based.
- It separates content from presentation.
- It supports nesting of elements to represent complex data.
- Tags are case-sensitive and must be properly closed.
- White spaces are preserved in XML.

Example:

```
<employee>
  <name>John</name>
  <id>101</id>
</employee>
```

Comparing XML with HTML

Although XML and HTML look similar, they serve entirely different purposes. HTML is a presentation language used to display content in web browsers, while XML is a data transport and storage language.

Feature	XML	HTML
Purpose	Store and transport data	Display data
Tag Definition	User-defined	Predefined
Closing Tags	Mandatory	Optional in some cases
Structure	Strict and hierarchical	Flexible

Data Handling	Carries data	Displays data
Syntax Sensitivity	Case-sensitive	Not case-sensitive

HTML Example:

`<p>This is a paragraph</p>`

XML Example:

`<paragraph>This is a paragraph</paragraph>`

Structure of XML

An XML document contains different components, each serving a specific purpose:

XML Declaration:

- Appears at the top of the file.
- Specifies XML version and encoding.

`<?xml version="1.0" encoding="UTF-8"?>`

Elements:

- Primary building blocks of XML.
- Each element must have an opening and closing tag.

`<name>John</name>`

Attributes:

- Provide additional information inside the start tag.

`<student id="101">John</student>`

Comments:

- Used to add notes within the XML document.

`<!-- This is a comment -->`

CDATA :

CDATA stands for **Character Data**.

In XML, CDATA is used to include text that should not be parsed by the XML parser. The parser treats the content inside CDATA as raw, plain text, even if it contains symbols that normally have special meaning in XML.

XML uses special characters like <, >, &, ", and ' for tags and attributes. These cannot be directly written inside XML content, or they will confuse the XML parser.

Syntax of CDATA

<![CDATA[

raw text with special characters goes here.

]]>

- Starts with: <![CDATA[
- Ends with:]]>
- The content inside is not processed or parsed by the XML parser.
- We cannot use]]> inside the CDATA content, as it is the ending tag.

Note:]]> cannot be used inside the CDATA content, as it is the ending tag.

Example:

```
<info><![CDATA["XML" is <awesome> & powerful]]></info>  
<text><![CDATA[5 < 10 & 10 > 2]]></text>
```

Entity References:

- Replace reserved characters like <, >, &, etc.

< < :less than

> > greater than

& & ampersand

' ' apostrophe

" " quotation mark

Example:

Ex: `<message>5 < 10 and 20 > 15</message>`

output: 5 < 10 and 20 > 15

Parsers:

Parsers are tools that read and process XML documents. There are two primary types:

a. DOM (Document Object Model) Parser

- Loads the entire XML document into memory and represents it as a tree structure.
- Each element, attribute, and text node becomes a part of this tree.
- Allows random access to any part of the document.
- Suitable for applications where the entire document needs to be accessed or modified.
- Memory-intensive for large XML files.

Example:

```
from xml.dom import minidom
doc = minidom.parse("students.xml")
students = doc.getElementsByTagName("student")
for s in students:
    print(s.getElementsByTagName("name")[0].firstChild.data)
```

b. SAX (Simple API for XML) Parser

- Reads the XML document sequentially from top to bottom.
- Does not load the entire document into memory.
- Triggers events (startElement, endElement, characters) while reading.
- Suitable for large XML files or when only parts of the data are needed.
- More memory-efficient but less flexible than DOM.

Example:

```
import xml.sax
class StudentHandler(xml.sax.ContentHandler):
    def startElement(self, name, attrs):
        if name == "student":
            print("Student ID:", attrs["id"])

parser = xml.sax.make_parser()
```

```
parser.setContentHandler(StudentHandler())  
parser.parse("students.xml")
```

Document Type Definition (DTD)

DTD defines the legal building blocks of an XML document, such as the elements and attributes allowed and their structure. It ensures the validity of the document.

Uses of DTD:

- Ensures data consistency.
- Validates XML structure.
- Helps applications interpret XML correctly.

Types:

There are two main types of DTD:

a. Internal DTD

An internal DTD is defined inside the XML document itself. It is embedded within the `<!DOCTYPE>` declaration at the top of the document. This method is suitable when the DTD is simple and used by only one XML file.

Example:

```
<?xml version="1.0"?>  
  
<!DOCTYPE student [  
  <!ELEMENT student (name, age)>  
  <!ELEMENT name (#PCDATA)>  
  <!ELEMENT age (#PCDATA)>  
]>  
  
<student>  
  <name>John</name>  
  <age>22</age>  
</student>
```

In the above example:

- The DTD is enclosed in square brackets `[]` within `<!DOCTYPE>`.
- It declares a student element that must have name and age.

- #PCDATA stands for parsed character data.

b. External DTD

An external DTD is defined in a separate file and linked to the XML document using the SYSTEM keyword. This is preferred when multiple XML documents use the same DTD, promoting reusability and centralized management.

Example:

student.dtd (external file):

```
<!ELEMENT student (name, age)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT age (#PCDATA)>
```

student.xml:

```
<?xml version="1.0"?>
<!DOCTYPE student SYSTEM "student.dtd">
<student>
  <name>John</name>
  <age>22</age>
</student>
```

In the above example:

- <!DOCTYPE student SYSTEM "student.dtd"> tells the parser to use the rules defined in student.dtd.
- This makes the XML valid as long as it follows the structure defined in the external file.

XSLT with XML

XSLT (eXtensible Stylesheet Language Transformations) is a language designed for transforming XML documents into other formats such as HTML, text, or other XML documents. It provides a way to apply logic and formatting rules using templates and expressions. XSLT is essential when XML data needs to be presented in a human-readable format or shared across different systems with required structure and style.

It is based on XPath for selecting parts of the XML document. XSLT is often used in real-world applications like report generation, dynamic content rendering, and converting XML to HTML webpages.

Basic Structure of XSLT:

```
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">  
  <!-- XSLT code goes here -->  
</xsl:stylesheet>
```

a. <xsl:template>

The <xsl:template> element is fundamental in XSLT as it defines how a specific XML node should be transformed. Each template matches a particular XML element or pattern using the match attribute.

- It serves as the main transformation logic.
- The root template usually matches / which is the root of the XML tree.
- Inside the template, we can place HTML tags or other XSL elements to format the output.
- Templates can be reused and applied selectively using other XSL instructions.
- Supports complex formatting and conditional logic.
- Multiple templates can be defined for different XML elements.
- Templates are matched using XPath expressions.

Example:

```
<xsl:template match="/">  
  <html>  
    <body>  
      <h2>Student Details</h2>  
      <xsl:apply-templates/>  
    </body>  
  </html>  
</xsl:template>
```

b. <xsl:apply-templates>

This element tells the XSLT processor to process child nodes of the current node using matching <xsl:template> rules.

- Enables recursion in templates.
- Helps separate formatting rules for different XML elements.

- Makes the stylesheet modular and maintainable.
- Can be controlled using select attribute.
- Works well with complex XML structures.
- Provides flexibility when combining with conditions.
- Improves code reusability.

Example:

```
<xsl:template match="students">
  <xsl:apply-templates select="student"/>
</xsl:template>
```

c. <xsl:import> and <xsl:include>

Both these elements are used to include stylesheets inside other stylesheets.

- <xsl:import>: Allows importing another stylesheet and its templates. Templates in the current file override imported ones.
- <xsl:include>: Simply includes the contents of another stylesheet. No override feature.
- Encourages modular design.
- Reduces code duplication.
- Improves manageability in large projects.

Example:

```
<xsl:import href="base-template.xml"/>
<xsl:include href="header.xml"/>
```

d. <xsl:element>

Used to create an element dynamically during transformation.

- Tag name can be created using variable or XPath.
- Prevents hardcoding of tag names.
- Useful in dynamic report generation.
- Helps when element names depend on data.

Example:

```
<xsl:element name="title">
```

```
XML Guide
```

```
</xsl:element>
```

Output:

```
<title>XML Guide</title>
```

e. <xsl:attribute>

Used to add attributes dynamically to an element within <xsl:element>.

- Used inside <xsl:element> only.
- Supports dynamic values using XPath.
- Allows conditional attribute assignment.
- Used in generating structured data like tables.

Example:

```
<xsl:element name="student">
```

```
<xsl:attribute name="rollno">101</xsl:attribute>
```

```
<xsl:value-of select="name"/>
```

```
</xsl:element>
```

Output:

```
<student rollno="101">John</student>
```

f. <xsl:value-of>

Extracts and outputs the value of a specific XML element or attribute.

- Accepts XPath expressions.
- Converts node content into string.
- One of the most frequently used elements.
- Cannot contain other elements.
- Common in text extraction scenarios.

Example:

```
<xsl:value-of select="student/name"/>
```

g. Conditional Statements**1. <xsl:if> – Conditional Display**

Used to perform conditional execution based on test expression.

Executes a block **only if** a condition is **true**.

- The test attribute uses XPath.
- Allows logic-based formatting.
- Can display or hide content.
- Does not support else; use <xsl:choose> for alternatives.

Syntax:

```
<xsl:if test="condition">
```

```
<!-- Content to show -->
```

```
</xsl:if>
```

Example:

```
<xsl:if test="marks >= 40">
```

```
<p>Passed</p>
```

```
</xsl:if>
```

2. <xsl:choose>, <xsl:when>, <xsl:otherwise> :

Works like if...else if...else

Use when you have multiple conditions.

Syntax:

```
<xsl:choose>
```

```
<xsl:when test="condition1">...</xsl:when>
```

```
<xsl:when test="condition2">...</xsl:when>
```

```
<xsl:otherwise>...</xsl:otherwise>
```

```
</xsl:choose>
```

Example:

```
<xsl:choose>
  <xsl:when test="marks >= 75">
    <p>Distinction</p>
  </xsl:when>
  <xsl:when test="marks >= 40">
    <p>Pass</p>
  </xsl:when>
  <xsl:otherwise>
    <p>Fail</p>
  </xsl:otherwise>
</xsl:choose>
```

3. Use with Loops

<xsl:for-each>:

It is used to loop over a set of nodes in the XML.

Similar to for or foreach loop in other programming languages.

Syntax:

```
<xsl:for-each select="path-to-nodes">
  <!-- What you want to do with each node -->
</xsl:for-each>
```

Example:

```
<xsl:for-each select="students/student">
  <xsl:if test="marks >= 40">
    <p><xsl:value-of select="name"/> - Pass</p>
  </xsl:if>
</xsl:for-each>
```

h. <xsl:sort>

Used inside <xsl:for-each> or <xsl:apply-templates> to sort nodes.

- Sorts in ascending/descending order.
- Can be based on text or numbers.
- Improves presentation in output.

Example:

01. <xsl:for-each select="students/student">

```
<xsl:sort select="name" order="ascending"/>
```

```
<xsl:value-of select="name"/>
```

```
<br/>
```

```
</xsl:for-each>
```

02.

```
<xsl:for-each select="students/student">
```

```
<!-- Sort by marks in descending order -->
```

```
<xsl:sort select="marks" data-type="number" order="descending"/>
```

```
<p>
```

```
<xsl:value-of select="name"/> -
```

```
<xsl:value-of select="marks"/> marks
```

```
</p>
```

```
</xsl:for-each>
```

Note: XSLT takes text by default in sorting, so use **data-type="number"** to sort numeric values correctly.

i. Common XSLT Functions

XSLT supports various XPath functions that enable advanced data manipulation and conditional logic during transformation. Below are some of the most widely used functions explained in detail with examples:

1. string-length()

This function returns the number of characters in a given string, including spaces.

Example:

```
<xsl:value-of select="string-length('Hello World')"/>
```

Output: 11

It's commonly used for validation, trimming, or formatting purposes.

2. contains(str, sub)

Checks whether the str contains the substring sub. It returns true or false.

Example:

```
<xsl:if test="contains(name, 'John')">  
  <p>Contains John</p>  
</xsl:if>
```

Displays the paragraph only if the name contains "John".

3. substring(str, start, len)

Extracts a part of the string starting from the start position with a specified len.

Example:

```
<xsl:value-of select="substring('HelloWorld', 6, 5)"/>
```

Output: World

4. concat(str1, str2)

Joins two or more strings into one.

Example:

```
<xsl:value-of select="concat(firstname, ' ', lastname)"/>
```

If firstname = 'John' and lastname = 'Doe', Output: John Doe

5. normalize-space(str)

Removes leading and trailing spaces and replaces multiple spaces within the string with a single space.

Example:

```
<xsl:value-of select="normalize-space(' XML Tutorial ')/>
```

Output: XML Tutorial

Useful when dealing with input that has inconsistent spacing.

6. starts-with(str, prefix)

Returns true if a string starts with a specified substring.

Example:

```
<xsl:if test="starts-with(name, 'A')">
  <p>Name starts with A</p>
</xsl:if>
```

7. not(condition)

Logical negation. Returns true if the condition is false and vice versa.

Example:

```
<xsl:if test="not(price > 100)">
  <p>Price is acceptable</p>
</xsl:if>
```

8. position()

Returns the current position of a node within a node-set, often used in loops.

Example:

```
<xsl:for-each select="student">
  <xsl:value-of select="position()"/>
</xsl:for-each>
```

Prints 1, 2, 3... based on the position.

9. last()

Returns the position of the last node in a node-set.

Example:

```
<xsl:if test="position() = last()">
  <p>This is the last item</p>
</xsl:if>
```

10. boolean()

Converts a given expression to a Boolean value (true or false).

Example:

```
<xsl:if test="boolean(name)">
  <p>Name exists</p>
</xsl:if>
```

This is useful for checking if a field is non-empty or exists.

Well-Formed and Valid Documents

A **well-formed XML document** follows all basic syntax rules:

- All elements must be properly nested.
- All elements must have a closing tag.
- Attribute values must be in quotes.
- Tags are case-sensitive.

Example of well-formed XML:

```
<school>
  <student>
    <name>Arun</name>
    <age>21</age>
  </student>
  <teacher>
    <name>Meera</name>
    <subject>Math</subject>
  </teacher>
</school>
```

A **valid XML document** is well-formed and conforms to a DTD or XML Schema.

Benefits of validation:

- **Ensures consistent structure:** Validation checks whether the XML follows a predefined structure (like DTD or XML Schema), making sure all required elements are present and ordered correctly.
- **Prevents data interpretation errors:** Validating XML helps catch missing tags, misnamed elements, and other issues early, reducing the chances of misinterpreting or corrupting the data during processing.
- **Makes the document suitable for system processing:** Systems and applications that consume XML data rely on its structure being predictable and valid. Validation guarantees that the document can be processed without unexpected runtime issues.
- **Promotes interoperability between systems:** When data is exchanged between multiple systems (e.g., in APIs or web services), validation ensures the sender and receiver agree on the data format and structure.
- **Improves data integrity and accuracy:** By enforcing rules on the types and values of data, validation helps maintain high-quality, clean, and accurate data.
- **Simplifies debugging and maintenance:** Developers can identify and fix structural issues faster when the document is validated, leading to more stable applications.